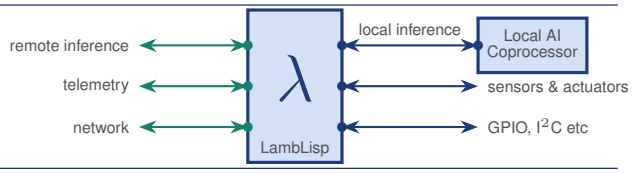
LambLisp

From AI to Actuator™



A Lisp Language System for the Full Control Stack

LambLisp is a standards-conforming Lisp language system for the full control and AI stack. The same unmodified source code runs on a Linux cloud server doing AI reasoning, an Nvidia Jetson Orin Nano running local neural inference, and an ESP32 microcontroller executing hard real-time control loops. LambLisp's innovative design eliminates translation and marshalling between these layers, reducing the semantic distance between AI reasoning and physical effect. Each layer exchanges information with the others in the same format as the executing code itself: LambLisp.

LambLisp nodes communicate with supervisory and cloud tiers over any duplex channel (TCP/IP, UART, etc) for remote evaluation, hot procedure reload, and telemetry — without restarting the device.

LambLisp can also orchestrate the Python and CUDA AI stacks, so the same language that operates the hardware also decides when to run inference and what to do with the result.

The execution engine provides three selectable tiers: an AST interpreter for rapid portability and efficient debugging, a bytecode compiler for compact, fast interpretation, and a native-code generator (NCG) that speeds execution further, activating automatically using adaptive heuristics. LambLisp's incremental GC interleaves collection in bounded time slices so control loops run deterministically.

Language standard	Scheme R7RS with real-time enhancements
Execution	Selectable: AST traversal, bytecode VM, or platform-optimized native code
GC	Adaptive real-time incremental mark-sweep
Numeric tower	Integer (int32, int64, bignum), rational, float32, float64, complex32
Filesystem	SPIFFS, LittleFS, POSIX
Protocols	I²C, SPI, WiFi, LLIP; PROFIBUS DP- and PROFINET-compatible[4]
Platforms	ESP32 (all variants), Nvidia Jetson Orin Nano, Linux x86-64, AArch64
C++ API	`Lamb` class; single standard function signature, no FFI required

Three-Tier Execution Engine

AST Traversal

- Constructs an Abstract Syntax Tree as 1st phase of compilation
- Evaluates the parsed AST directly; no code generation phase
- Fast pointer threading: direct dispatch to compiled C++ procedures with no interpreter overhead at the boundary
- Tree-walking evaluator with trampoline for tail calls
- Proper tail recursion per R5RS §3.5
- Full debug information retained: variable names, source structure, and expression context available at runtime

Bytecode VM (BC)

- Extends AST traversal with custom bytecode and inline primop expansion
- Typically 2–5× faster than AST traversal; list and loop kernels 10–100×
- Example: sieve(200) 115× over AST on ESP32 LX6 (BC 4.8 ms vs AST 558 ms)

Native Code Generator (NCG)

- Smart JIT compiler: activates automatically based on heuristic
- Direct NCG→NCG dispatch; no VM re-entry
- Backends: x86-64, AArch64 (Jetson), Xtensa (ESP32, ESP32-S3)
- IRAM placement for hot paths on Xtensa
- AST, BC, and NCG closures interoperate transparently

Adaptive Incremental Garbage Collection

LambLisp implements a real-time adaptive incremental garbage collector, extending the work of Dijkstra et al. (1978)[1] and Yuasa (1986)[2] with tunable GC time slices, adaptive collection triggers, and idle-time amortization.

The LambLisp GC is instrumented for runtime adaptability. LambLisp interleaves GC work with real-time tasks in bounded time slices; execution is never blocked for an unbounded pause.

Mark and sweep quantum times are calibrated, and idle-time amortization shifts GC work from high CPU demand to low, helping to ensure uniform, low-jitter loop times.

After each GC cycle, the trigger threshold and minimum heap size are recomputed from the live-cell count; the memory pool expands as needed to maintain real-time performance.

Security

LLIP transport is secured with RSA-2048 key exchange, Diffie-Hellman ephemeral key agreement, and AES-GCM authenticated encryption. All cryptographic operations use bignum arithmetic implemented in LambLisp's numeric tower.

On the ESP32, AES and SHA operations use the on-chip hardware acceleration engines, reducing cipher overhead and freeing the application core for control work.

Scheme's minimal, formally-specified grammar also reduces exposure to Trojan-source attacks, and makes LambLisp programs more amenable to automated formal analysis and correctness

proofs than equivalent C++ or embedded Python code.

Scheme R7RS Compliance

§3.5	Proper tail recursion	✓
§4.1–4.2	Core syntax and derived forms	✓
§4.3	Hygienic macros (`syntax-rules`)	✓
§6.1	Equivalence predicates	✓
§6.2	Numeric tower	✓
§6.3–6.8	Booleans, pairs, symbols, chars, strings, vectors	✓
§6.9	Bytevectors	✓
§6.10	Control features	✓[3]
§6.11	Exceptions (`guard`, `raise`)	✓
§6.13	I/O	✓

Real-Time and AI Extensions

Typed vectors (n-dimensional arrays)	✓
I²C, SPI, PWM, GPIO (embedded)	✓
WiFi / TCP/IP	✓
CUDA inference integration (Jetson / x86-64)	✓
LLIP remote evaluation and hot-reload	✓
PROFIBUS DP-compatible slave (RS-485)	uncert.[4]
PROFINET-compatible IO-Device (DCP)	uncert.[4]
Interrupt-driven GPIO (embedded)	✓
Advanced real-time garbage collector	✓
Arbitrary-precision integers (bignum)	✓
RSA-2048 / DH + AES-GCM encryption (LLIP)	✓
Heap auto-persistence at boot	✓
Serial REPL over USB / UART (embedded)	✓
Native CRC-32 over bytevectors (`crc32`)	✓

Numeric Tower

Numbers are promoted or demoted automatically to the most compact and accurate type that preserves value.

Exact integer 32	32-bit exact; immediate in pair cells
Exact integer 64	64-bit exact
Bignum	Arbitrary-precision exact; used for RSA and DH cryptography
Exact rational	32+32-bit; num/den pair; GCD-reduced
Single float	32-bit IEEE 754; inexact; immediate
Double float	64-bit IEEE 754; inexact
Complex	32+32-bit; single-precision real/imaginary

Real-time integer handling: exact 32-bit integers are stored immediately in pair cells with the type tag in the cell's flag byte — the full 32-bit range is native, with no value bit sacrificed to a pointer tag. Full-width quantities common in embedded control — microsecond timestamps, 32-bit hardware registers and bit-masks, and CRC-32 checksums — stay native and allocation-free, avoiding the heap-bignum promotion (and the stop-the-world GC pauses it triggers) seen with tagged-fixnum representations. The native `crc32` primitive computes IEEE-802.3/zlib CRC-32 over a bytevector slice.

C++ Integration

LambLisp Operator Signature

```
Sexpr_t fn(Lamb &lamb,
           Sexpr_t sexpr,
           Sexpr_t env_exec);
```

Register via `mk_Mop3_procst_t` / `dict_bind_bang`.

Example: NeoPixel Driver

```
// (neopixel-write pin r g b)
Sexpr_t mop3_neopixelWrite(
    Lamb    &lamb,
    Sexpr_t  sexpr,
    Sexpr_t  env_exec) {
  LL_int32 pin = lamb.car(sexpr)->mustbe_int32();
  LL_int32 r   = lamb.cadr(sexpr)->mustbe_int32();
  LL_int32 g   = lamb.caddr(sexpr)->mustbe_int32();
  LL_int32 b   = lamb.cadddr(sexpr)->mustbe_int32();
  neopixelWrite(pin, r, g, b);
  return OBJ_VOID;
}
```

Example: Extending for Speed

Any hot function can move to C++ with no FFI and no wrapper layer. The library's own `iota` is written this way: claim the list spine in one bulk request, then fill it in a single forward pass.

```
// core of (iota n); argument
// parsing omitted
Sexpr_t head =
  lamb.bulk_alloc_pairs(count, env_exec);
lamb.gc_root_push(head);
Sexpr_t p = head;
for (LL_int32 i = 0; i < count; i++) {
  Sexpr_t next =
    p->prechecked_anypair_get_cdr();
  p->rplaca((Word_t)
    lamb.mk_integer(istart + i*istep,
                    env_exec));
  p = next;
}
lamb.gc_root_pop();
return head;
```

The same function written in Scheme, building a 500-element list on ESP32-S3, each measured in a fresh boot over a fixed window:†

Implementation	µs	vs C++
Scheme, AST	140,027	62×
Scheme, bytecode	8,152	3.6×
Scheme, native	2,216	1.0×
C++ builtin	2,269	—

Moving the function into C++ buys 62× over interpretation. Note the last two rows: for allocation-shaped work the native code generator already *matches* hand-written C++, so C++ is the tool

for hardware access and library reuse rather than an automatic speed win.

Bulk allocation needs its cells at once: if fewer are free the collector runs to completion first. An application that must bound that pause queries (`Platform.nfree`) and either sizes the request to fit or builds incrementally, where each `cons` pays one bounded quantum.

† Fresh boot per row, mean over a fixed 3 s window: the collector's `Amax` high-water mark is monotonic, so measuring several variants in one session charges the later ones for the earlier ones' peak.

LLIP — LambLisp Interaction Protocol

LambLisp Interaction Protocol (LLIP) over TCP/IP:

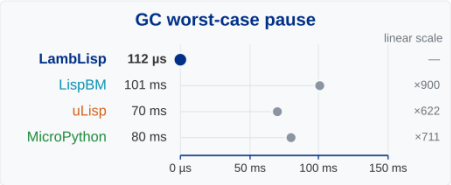
- Remote `eval` with result streaming
- Hot-reload: redefine procedures without restart
- Mutual authentication via RSA-2048
- Session encryption via DH + AES-GCM

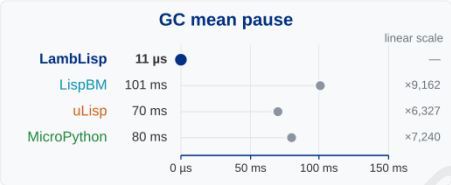
Notes

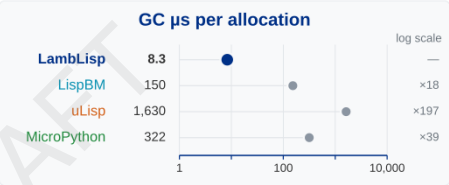
1. Dijkstra, E.W. et al., "On-the-fly Garbage Collection: An Exercise in Cooperation," *Commun. ACM* 21(11):966–975, 1978.
2. Yuasa, T., "Real-Time Garbage Collection on General-Purpose Machines," RIMS TR-86-17, Kyoto University, 1986; *J. Syst. Softw.* 11(3):171–179, 1990.
3. `call-with-current-continuation` is not supported. Full continuations require capturing and preserving the entire call stack for later reuse. LambLisp obtains a performance advantage without `call/cc`. Functions that depend on `call/cc`, such as `dynamic-wind` and `parameterize` are also not supported.
4. **PROFIBUS DP- and PROFINET-compatible; not certified.** The implementations are complete and under test — an RS-485 DP slave, and an IO-Device answering DCP identify/set so a controller can commission the node by name — and ship in the next release. They have *not* been certified at an accredited PI Test Lab, and no conformance class is claimed. A Vendor ID and a matching GSD/GSDML file are required before a device is offered as a PROFIBUS or PROFINET product. PROFIBUS® and PROFINET® are registered trademarks of PROFIBUS Nutzerorganisation e.V. (PI); they are used here to state interoperability only, and imply no endorsement or certification by PI.

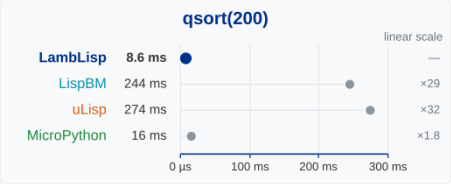
GC Pause Latency and Execution Speed

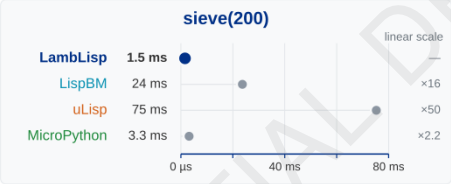
LambLisp's incremental GC bounds the per-allocation pause to one mark or sweep quantum, independent of live heap size. The stop-the-world collectors must scan the entire live set on every collection, so their worst-case pause grows with the heap.

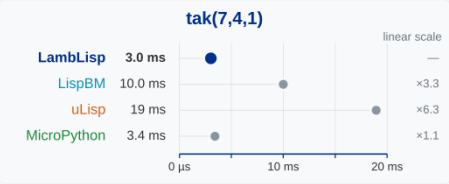


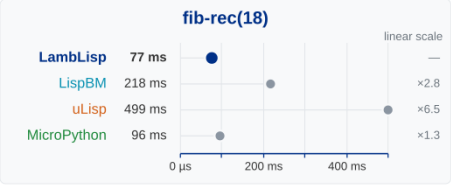


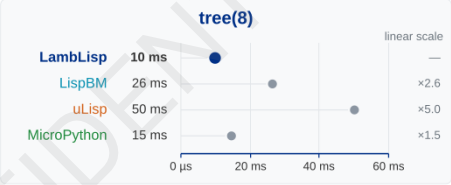


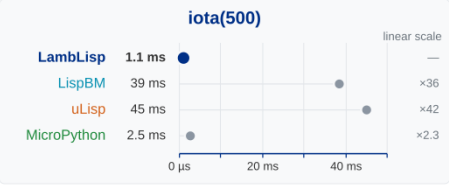


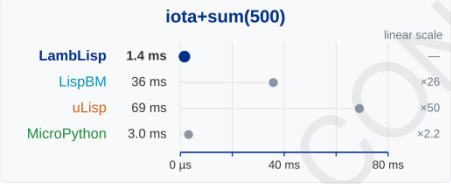


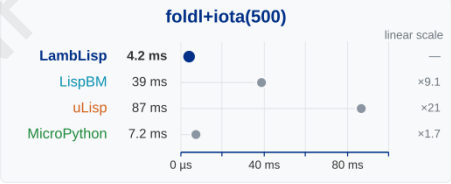












Test Conditions

ESP32-S3 rev v0.2 · 2 MB PSRAM · dual-core Xtensa LX7 · 240 MHz

GC Test Conditions

Common protocol: 30,000-object live set and 30 collections per runtime, on the same board.
LambLisp: gc-pause-bench; + ~10,209-cell prelude (live = 40,209); 12,288-cell block × ≤18, PSRAM; incremental — 1,206,270 allocs
LispBM: build-list live set; 64 KB DRAM heap; incremental GC — 20,184 allocs
uLisp 4.9a: churn carried in globals only (no let bindings); 260K-object PSRAM heap — 1,281 allocs
MicroPython 1.24: live set of small tuples; forced full gc.collect() — 7,421 allocs

Performance Test Conditions

LambLisp: NCG; PSRAM cell heap; ESP32-S3 dual-core LX7 240 MHz
LispBM: master 2026-05-20; 64 KB DRAM heap; incremental GC
MicroPython: v1.24.1; SPIRAM enabled
uLisp: 4.9a; 260K-object PSRAM heap; Common Lisp

Active Research Areas

1. **Industrial controls** — Distributed sensor/actuator nodes report to a SCADA-level LambLisp supervisor via LLIP. Updated control logic is pushed live as new code — no firmware update or node reboot required. Continuous telemetry enables automated monitoring and early fault detection.
2. **Medical and assistive devices** — LambLisp's bounded GC pause makes it suitable for safety-critical embedded devices: infusion pumps, powered mobility aids, and environmental sensors. Treatment protocols and device policies are updated from a clinical or fleet supervisor via LLIP without reflashing.
3. **Autonomy and cooperative behavior** — LambLisp nodes receive motion instructions as executable code — waypoints, tool paths, formation commands — and translate them directly into hardware control. A Jetson edge node performs SLAM or mission planning via CUDA and pushes updated instructions to mobile nodes via LLIP at any time. Applies to multi-axis arms, mobile robots, and aerial and underwater vehicles.
4. **Precision agriculture** — LambLisp sensor nodes in the field report to a Jetson running crop-management logic. CUDA-accelerated imaging drives irrigation and treatment decisions; updated schedules are pushed to actuator nodes as new code.
5. **Aerospace** — Attitude control with GC guarantees; maneuver strategies uploaded as new LambLisp expressions via LLIP uplink. Guidance algorithms promoted to native code for deterministic timing; destination and engagement parameters updated as new instructions via encrypted LLIP before or during flight.

Licensing and Availability

Free for non-commercial use. Commercial use requires a license.

Contact: Clive Smith
`clive.smith@frobeniusnorm.com`
+1 978 566 2300

Full documentation is available at `lamblisp.com`

Frobenius Norm LLC
`clive.smith@frobeniusnorm.com`
`www.frobeniusnorm.com`
+1 978 566 2300